

Large Pattern Models: Replacing Token Consumption with Artifact Reuse

iulian dacineu

unia — <https://github.com/dacineu/unia>

September 2026

Abstract

Language models are consumed per request. A coding agent that has solved a task before pays for it again, because the solution it produced is stored as *text* — in context, in a commit, in a skill file — and text must be re-read by a transformer to become an action. This paper argues that the unit of reuse should be an *executable artifact* rather than a *description*, and that the resulting system should be optimised for a metric we call **escalation rate**: the fraction of requests that cannot be served from a local artifact and must reach a provider.

I formalise a resource model, the `.ure` format, in which a unit of capability is a content-addressed declaration bound to a runnable body. I describe an induction loop that converts recorded interactions into candidate artifacts without gradient descent, an evidence gate that makes premature promotion structurally difficult, and a collection protocol in which each artifact observes itself. The prototype is *unia*, in Rust, with a Model Context Protocol server surface.

Measurements characterise the retrieval tier and establish a first retrieval-quality baseline: a $4.6\mu\text{s}$ lexical matcher achieves 90% Hit@1 against a hand-written fixture, with all observed failures attributable to manifest authoring rather than to a need for semantic machinery. I am explicit that this fixture is too small to support the thesis, and I state the experiment that would falsify it.

1 Introduction

The dominant cost of a language-model-backed agent is not reasoning. It is re-deriving knowledge the system already possesses.

Consider a coding agent asked to add a depen-

dency and refresh a lockfile. The second time it is asked, the answer is identical. The first answer may have taken 2,000 input tokens and 600 output tokens. The second costs the same, because the first answer was recorded as *text* — a diff, a file, a markdown skill — and text is inert. It becomes an action only by passing through a transformer again.

Three mechanisms mitigate this, and all three leave the architecture unchanged:

Mechanism	Eliminates
Prompt caching	input cost on a repeat
Semantic caching	the request, on near-match
RAG	nothing; it adds context

The ceiling on all three is set by the same thing. A cache can only return something previously computed and stored. If the stored artifact is a *description* of an action, serving it requires a model. If the stored artifact *is* the action, serving it does not.

1.1 The claim

C1. For a workload with sufficient repetition, a system that stores executable artifacts rather than descriptions can drive escalation rate toward zero without gradient training, and per-served-request token cost toward zero, because the model is *out-routed* rather than *distilled*.

“Sufficient repetition” without training is a deliberate constraint, not a limitation: the learned artifact is inspectable, auditable, and deletable.

1.2 What would falsify C1

C1 fails if, on a labelled set of real coding intents, a pattern corpus of realistic size (200–2,000 artifacts)

fails to route a large majority of requests — that is, if a substantial residue of intents shares no vocabulary with any candidate artifact and resists paraphrase. I do not have this experiment. The prototype exists to make it answerable, not to claim C1 is established.

2 Prior work and my relation to it

This is not a new idea in the shape of the idea. It is a specific claim about the *unit* of reuse, and the literature contains most of the ingredients.

Skill induction. Work establishing that agents which memorise action sequences fail, and that inducing *rules* — explicit control flow, preconditions, variable binding — generalises better (NSI, ICML 2026). SkillGen synthesises a single auditable skill from trajectories. SkillRevise couples execution evidence with general repair knowledge. My induction follows this line: traces are grouped by the **primitive sequence**, and the observed phrasings become aliases, so the induced unit is a rule and not a script.

Executable accumulation. AgentFactory accumulates executable subagents and reuses them. This is the closest prior work to my claim, and I claim no novelty over it.

Library learning and program synthesis. DreamCoder-style systems induce reusable libraries from traces. The difference is the target: a synthesised program versus a declaration of intent bound to a pre-existing driver.

Semantic caching. Caches return prior *responses*. Here the cache entry is executable and the provider is not called. The relationship is a strict generalisation.

My specific position is the combination of three properties: the reusable unit is **content-addressed**, so two independently induced rules that are textually identical collapse to one artifact without a comparison step; learning is **zero-gradient**, producing inspectable artifacts rather than weight deltas; and the objective is **escalation rate** rather than accuracy, because accuracy is the wrong metric when a correct answer still costs a request.

3 The resource model

Following `docs/ure-specification-formal.md`, a resource is a tuple

$$\mathcal{U} = \langle \mathcal{I}, \mathcal{S}, \mathcal{A}, \mathcal{C} \rangle$$

where $\mathcal{I}$ is identity, $\mathcal{S}$ a state space, $\mathcal{A}$ a set of action primitives, and $\mathcal{C}$ a set of constraints. Identity is a content address: a DU-UUID derived from the canonical serialisation of the manifest with `resource_id` removed, hashed with SHA-256. Consequently a resource cannot contain its own identity in its identity computation, and two manifests that serialise identically *are* the same resource.

3.1 The gap between the formalism and the code

The formal specification requires that constraints “must return **True** for an action to be dispatched to the Nucleus.” The implementation prints constraints for operator visibility and does not evaluate them. This is recorded as divergence D5 and is not closed. I report it because it is load-bearing: Section 6 assumes constraints gate dispatch, and today they do not. The argument therefore rests on the evidence gate alone.

Two identity divergences also stand: the identifier scheme documented in prose is not the one implemented (D2), and a document category vocabulary and a resource-class enumeration are both in use for the same dimension (D8).

4 Architecture

Tier	Behaviour
1	exact / containment / IDF overlap; no model
1.5	semantic re-rank of the residue; optional
2	verified artifact executes; 0 tokens
3	escalate to a provider; full cost

The ladder’s property is not accuracy but **monotone improvement**: every promotion moves traffic from a paying tier to a free one, and the model is retired by being out-routed rather than replaced. Tier 1 requires no model at all, so the architecture is useful before any inference engine exists.

4.1 Retrieval

Tier 1 scores every candidate phrase against the intent using inverse-document-frequency-weighted token overlap, with a decisive fast path for containment. Trigram similarity is deliberately not used: it ignores term frequency, and for short technical intents — where one distinctive token carries most of the signal — that is strictly worse.

Measured, release build, single machine:

Corpus	p50	p99
12	4.6 μ s	6.3 μ s
1,000	405 μ s	644 μ s
5,000	2.17 ms	3.85 ms
20,000	9.96 ms	11.2 ms

Scaling is linear at approximately 0.5 μ s per artifact, which is the limitation of a scan rather than of an inverted index. For comparison, SQLite FTS5 with a real inverted index returns in 48 μ s at 50,000 artifacts in an 18.6 MB file. Retrieval, not induction, is the component that breaks first at scale.

4.2 Storage as a view, not a copy

The corpus is authoritative: a tree of `.ure` files under version control. The analytical index is a **view over those files** rather than a copy, so an edit in Git changes query results immediately and there is no migration to write. Only mutable state — promotions, traces, observations — is a table. This dissolves the consistency problem an ingest pipeline introduces, at the cost of requiring the corpus to be schema-consistent, which it is not yet.

5 Induction: learning without gradients

`src/induce` converts recorded interactions into candidate artifacts. Traces are grouped by the **primitive sequence** the intent resolved to, not by phrasing. Two sentences that reduced to the same primitives are evidence for one rule; the sentences themselves become the action’s aliases. A trace with no recorded sequence is skipped rather than guessed at.

For each group the module synthesises a candidate with an action id equal to the primitive sequence, stable across phrasings by construction; aliases drawn from every distinct observed phrasing, plus both underscore and space forms; constraints recording the observed failure rate, so a precondition is learned rather than re-paid for on every call; and a `resource_id` stamped with the DU-UUID of the resulting body, so identical rules deduplicate by content address.

It is not fine-tuning, and no component learns weights. Two consequences follow, both intended: training requires no accelerator, being string and integer work over a trace log; and the learned artifact

is a readable file, so a wrong artifact can be deleted and its deletion is a commit.

6 Safety

The characteristic risk of a system that promotes learned artifacts is that **a wrong artifact becomes fast, silent and reused**. A slow wrong answer is recoverable. A fast wrong answer served from a cache and re-derived from a trace is not, because the trace then confirms it.

The prototype treats this as the design constraint rather than a downstream check:

- Candidates are emitted as **candidate** and never as **champion**. Promotion is a separate, deliberate step; a function capable of emitting a promotion would have no gate at all.
- A minimum of distinct observed phrasings is required. One observation is an anecdote, and the same sentence repeated ten times is one observation, not ten.
- A maximum failure ratio blocks promotion. A sequence that usually fails is not a rule.
- Failures become recorded preconditions rather than being discarded.
- Traces without a recorded primitive sequence are not inducted from.

Confidence is multiplicative in breadth and reliability, and log-scaled in the number of distinct phrasings so that it never saturates at the minimum. A straightforward `observations / minimum`, clamped to 1, reads 1.0 the instant the minimum is met, which would make the fourth distinct phrasing worth exactly as much as the second; this was found by a test, not by inspection.

6.1 A known weakness

The evidence gate raises the cost of promotion; it does not establish correctness. The evaluation reports a false-positive rate of 2 in 8 negatives, and D5 means constraints do not yet gate dispatch. The honest summary is that the system currently prevents *thin* evidence from being promoted, and does not yet prevent *wrong* evidence from being promoted.

7 Prototype evaluation

A labelled fixture of 30 hand-written queries, split 20 expected-hit and 10 expected-negative. The corpus is the two curated manifests; the other ten are quarantined and not indexed.

Metric	Value
Hit@1	18/20 — 90.0%
Hit@3	18/20 — 90.0%
False negatives	2
False positives	2
Unsatisfiable	0

All four failures are manifest-authoring defects rather than semantic failures. Three follow from aliases that are reasonable for their own action (`open file`, `get content`, `create document`) and actively harmful as resource-level match keys. The fourth requires either constraint evaluation or semantic scoring.

One failure is not noise and is worth stating separately. IDF is currently computed per *phrase* across the corpus, so a resource listing `valve` across many aliases has its own strongest term down-weighted for appearing in its own alias set; a single rare term elsewhere then wins. The corpus should be weighted per resource. This is unfixed, and it means the reported 90% is a floor rather than a current best.

7.1 Cost against a local language model

For the routing question specifically — *which artifact matches this intent?* — measured on the same machine:

System	Latency	Corpus
Qwen3-1.7B (128p+64o)	~2,860 ms	n/a
Tier 1 matcher	4.6 μ s	12
Tier 1 matcher	9.96 ms	20,000
Tier 1.5, bge-small	8.3–20.8 ms	any
SQLite FTS5	48 μ s	50,000

The language model is roughly 6×10^5 slower than tier 1 at the current corpus size. That comparison is not close, and it is also not the interesting claim, because tier 1 does strictly less work. The interesting comparison is on *cost per served request*, where tiers 1 and 2 are free and tier 3 is not.

7.2 Hardware

All figures come from one machine: AMD Ryzen 7 5700U (Zen 2, 8c/16t), AMD Radeon Vega 8 under Vulkan 1.4 (RADV RENOIR), 8 CUs, **no matrix cores**, sharing 15 GiB system RAM; reported by the backend as `matrix cores: none, int dot: 0`. This is a low-end mobile integrated GPU, reported in full because throughput without hardware is not interpretable, and because the absence of matrix cores is why decode throughput is roughly an order of magnitude below published figures for the same model class on discrete GPUs.

Embedding tier, bge-small-en-v1.5 Q4_K_M, 33.21M parameters, 22.93 MiB: 967 t/s at 8 tokens, 3,032 t/s at 32, 6,158 t/s at 128. Generation tier, Qwen3-1.7B: 272.8 t/s prompt processing and 26.8 t/s generation on the iGPU against 155.1 and 22.8 t/s on CPU.

8 Collection is distributed

The first implementation of degradation kept every artifact’s usage in a supervisor-side table and ran a global planning pass. That is a *reporting* architecture, not an inference one, and it was wrong for this system: `ActuatorDriver` exposes `execute()`, so an artifact is itself the observation point. Routing that to a supervisor to obtain a decision is strictly more work than deciding locally.

It also destroys a signal. An artifact attached to a nucleus that has gone away is, to a central collector, indistinguishable from one that is genuinely idle.

The corrected design gives each artifact a self report — connection, hit and miss counts, distinct observations, last use, and reachability — and a decision function of exactly that report and a policy. No corpus argument, no central usage table; the signature asserts the property.

8.1 What distribution makes mandatory

Moving the decision into the artifact exposed something the centralised version had hidden: an artifact can have no usage because it is **unwanted**, or because it is **unreachable**. Those require opposite responses.

The prototype already produces the second case. In the baseline, `write output to results.txt` matched nothing, because the action id is `write_file` and containment requires the

literal `write file`. That pattern is not idle. It is mis-indexed, and it reports zero usage indefinitely. A collector that cannot distinguish the two retires precisely the artifacts that need repair, and does so silently, because the symptom of a routing defect and the symptom of an unwanted artifact are the same observation.

Reachability is therefore a three-valued discriminator, and the third value is the important one:

State	Meaning	Action
Verified	a query matched it	normal retirement
Unreachable	queried; no match	quarantine
Unknown	never matched	never retire

Unknown is the default, deliberately. Absence of a match is not evidence of absence of demand, and most artifacts in a young corpus are in that state; defaulting to **Unreachable** would empty the corpus within one collection cycle.

Connection is authoritative for liveness and is not derived from lifecycle, because the two are independent facts: a candidate that lost its nucleus is served by lifecycle and detached by connection, and a harvested artifact a node wired up is unverified but connected. Deriving one from the other made both states unrepresentable and produced two sources of truth that could disagree.

Copies of one artifact disagree, because each observes only its own window. The reconciliation rule is **monotone toward the less destructive action**: a proposal to retire loses to evidence of use, always. A node with a narrow window therefore cannot retire a capability a node with a broad window is actively serving, and disagreement cannot compound. This is CRDT semantics applied to artifact lifecycle, and the novelty claim is explicitly disclaimed.

Nothing about observation is centralised, but the evidence gate remains the precondition for promotion. Distribution of observation does not require distribution of trust: a mesh in which any attached node can synthesise and promote arbitrary executables is precisely the attack surface described next.

9 Exchange, redundancy, and safety

9.1 Structural deduplication, and its preconditions

Exchange between models is a set union over content addresses, and is therefore idempotent and commutative: two models that independently induce the same rule exchange one artifact, not two. Redundancy is eliminated by the identity function rather than detected after the fact. This holds only if three properties are true, and one currently is not:

Property	Status
Identical body $\rightarrow$ identical address	holds
Canonical serialisation byte-stable	by accident
Version label correct	fails : labelled v4

A v4 label is the worst available choice, because RFC 4122 defines v4 as *random*, and tooling that reasons about v4 semantics will mishandle these identifiers. It should be v5, or a custom v8.

The canonicalisation property holds only by accident. The implementation comment asserts that `serde_json` does not guarantee key order; that comment is wrong. Without the `preserve_order` feature, `serde_json::Map` is `BTreeMap`-backed and the serialisation is already sorted. If any contributor enables `preserve_order` for an unrelated reason, the comment becomes true and every content address in existence changes with no error. This requires a test asserting address stability, not a comment.

9.2 The corpus is an injection surface

The characteristic risk of promoting learned artifacts is that a wrong artifact becomes fast, silent and reused, and Section 6 addresses it from the inside. Exchange adds a second path to the same failure, because a promoted artifact carrying a payload *is* a driver. A pattern that reaches champion is persistent local code execution, and exchange is therefore a remote code execution vector unless gated.

This is not hypothetical. koi Research reported ClawHavoc: 341 malicious skills found by the bot they were targeting, rising to 824 by February 2026. unia’s exposure is worse than a typical skill library’s, because `ActuatorDriver::execute` is called by the Nucleus.

Four gates follow from structures already in the design: a `payload_kind`: "none" safety label, so the capability declaration and the safety declaration become one field; re-verification of evidence on import, never inheritance of a peer's; content addressing as integrity proof, so a received manifest can be verified byte-identical to what the sender claims; and mandatory lineage via `source_external_id`.

There is a further exposure the skill-library literature does not discuss. A manifest's `guidance` and `aliases` are free text. If any part of a manifest is ever placed in a model's context — which is what a pattern-aware agent would do — then an attacker who controls the corpus controls part of the prompt. That is prompt injection arriving through the retrieval store rather than through the user.

9.3 Union is not safe when precision binds

The natural design is to merge pattern stores as a set union. The evaluation shows why that is wrong for this system. Precision is the binding constraint at 2 false positives across 10 negatives, and precision *degrades* as the corpus grows, because every alias is an additional opportunity to match the wrong artifact. Accepting a peer's patterns adds aliases faster than it adds coverage.

So a model can get **worse** by receiving patterns, and union is harmful exactly when it appears most attractive. Exchange must be evidence-weighted: artifacts are exchanged *with* their observation count, failure ratio and provenance; the evidence gate is re-run on import rather than trusted; and provenance is mandatory. This is a negative result, and the most useful claim in this section, because it contradicts the obvious design.

Three further mechanisms are needed beyond content addressing. Aliases should be edges in a graph, `alias` $\rightarrow$ (artifact, action), rather than fields owned by an artifact: two artifacts can then share an alias without duplicating it, and the alias is scoped to an action rather than a resource, which is the direct cause of two of the four failures above. Patterns should compose over other patterns rather than restate their steps. And promotion should be monotonic — if a champion already covers a capability, induction refines it rather than creating a sibling, or every re-induction cycle produces near-duplicates differing only in aliases, which never dedupe.

9.4 Interoperability, by severity

The constraint grammar is undefined: `constraints` is an array of free strings, two implementations will parse it differently, and D5 means neither evaluates it. You cannot execute a constraint you cannot parse, so this blocks cross-model execution outright. It is a specification task, not an engineering one.

The remaining gaps are degraded mode rather than blockers: no capability declaration, so a receiver cannot be told which of the 20 primitives it implements; no schema negotiation, since `ure_version` is decorative and nothing reads it; the DU-UUID version mislabelling above; unspecified canonical serialisation, with one code comment wrong about it; ambiguous alias semantics, resource-level or action-level, which is the scoring bug; and no manifest JSON Schema, so nothing validates an import.

10 What is not demonstrated

I consider this section load-bearing rather than a disclaimer.

- **No escalation rate is measured.** The corpus holds two usable artifacts. A rate over two artifacts is not an escalation rate.
- **The fixture is 30 queries authored by the system's author.** It is a smoke test. It found a real defect in an afternoon, which is its purpose, but it cannot estimate behaviour on real traffic. A 200-query fixture written by someone who did not build the matcher is the minimum credible next step.
- **No learned component has run.** The induction module is implemented and covered by tests, but no trace log is being written: `unia_record` exists and no agent calls it.
- **Collection has never run.** The policy is tested against synthetic usage and has never been reconciled against a real disagreement, so the monotonicity argument is unvalidated.
- **Constraints do not gate dispatch (D5).**
- **The analytical index is non-functional.** `term_idf` in the DuckDB retrieval layer raises a type-coercion error; the corpus views load and are verified, the ranking path is not.

- **The corpus is not schema-consistent.** Twelve artifacts carry roughly a dozen distinct key sets, one retains a corrupted key from an earlier substitution defect, none carries a human-readable title, and two use human-assigned identifiers where the specification expects a UUID.
- **No comparison against a learned router exists,** because no learned router exists.
- **The four failures above are unfixed,** so the reported 90% is a floor.

11 Threats to validity

The author wrote both the fixture and the matcher, which is a smoke test that found a real defect but is not an unbiased evaluation. $n = 2$ artifacts, so everything measured is at a corpus size with no statistical meaning. The safety argument has one pillar, since D5 leaves constraints ungated. No learned component has run against real usage, and its behaviour on adversarial or degenerate traces is unknown. The prior-art claims name the closest work I am aware of; a reviewer with better recall may find more, and I would expect a non-trivial amount of it. The prototype is 45 tests and one fixture, which is evidence of internal consistency, not external validity. Every figure comes from a Vega 8 with no matrix cores, the least favourable device in the class; the relative claims should transfer, the absolute latencies will not.

12 Contributions

Stated conservatively, relative to the prior work surveyed above.

1. A formalisation of the reusable unit as an executable, content-addressed artifact, with the property that identical induced rules deduplicate without comparison and renaming does not change identity.
2. A zero-gradient induction mechanism deriving both rule and alias set from recorded interactions, with an evidence gate specified to resist promotion on thin evidence.

3. Escalation rate as the primary objective, with the argument that accuracy is the wrong metric when a correct answer still costs a request.
4. A three-valued reachability discriminator arising from the observation that collection must be distributed, and a retirement protocol that is monotone toward the less destructive action.
5. A negative result on exchange: content-addressed union is unsafe when precision binds, so a model can degrade by receiving patterns.
6. A measured characterisation of the lexical and semantic tiers on low-end integrated hardware, including the observation that a scan is adequate to $\sim 10^4$ artifacts and an inverted index is required beyond it.
7. A first retrieval-quality baseline locating current failure modes in manifest authoring rather than retrieval, and identifying a document-frequency weighting defect that is cheap to fix.

I do not claim novelty over executable skill accumulation, a working learned model, a measured escalation rate, novelty in applying CRDT semantics to artifact lifecycle, or that the safety argument is complete.

13 Future work, in dependency order

1. Fix the manifest defects surfaced by the evaluation — scope aliases to actions, add the space forms, and correct document-frequency weighting to the resource level. Expect this to move the fixture to 100% and to establish how much of the remaining problem is authoring rather than retrieval.
2. Close the induction loop: write real traces, induce, re-measure. This is the first end-to-end exercise of the learning loop and the first test of whether induced aliases beat authored ones.
3. Falsify or support C1 with a fixture of 200 or more intents authored independently, reporting escalation rate and false-positive rate.
4. Define the constraint grammar, so constraints gate dispatch and the safety argument gains its intended second pillar.

5. Only then evaluate a semantic tier, and only on the residue that survives the first three steps. If the residue is small, the correct decision is to stop.
 6. Define what a correct pattern is. Every step above assumes an oracle for correctness; self-evaluation without a verifier is the unsolved part, and property-based testing against observed outcomes is the current approach.
- [9] *unia* specification and divergences: docs/SPEC.md, docs/ure-specification-formal.md.

Authorship and assistance

This paper is a single-author work by *iulian dacineu*, authored and researched with AI assistance. The author line uses a pseudonym. An earlier project by the same author, *eunia* — European Union National Informative Assistant — was submitted to the European Commission’s EUvsVirus Hackathon on 24 April 2020, and is cited by its canonical URL; the present work is a distinct design and not a continuation of it, although the distributed and multidimensional framing originates there. The prototype is MIT licensed; commercial enquiries are directed through LICENSE-COMMERCIAL.md.

References

- [1] Lifting Traces to Logic: Programmatic Skill Induction with Neuro-Symbolic Learning for Long-Horizon Agentic Tasks. ICML 2026.
- [2] SkillGen: synthesis of auditable skills from trajectories via contrastive induction.
- [3] SkillRevise: trace-conditioned skill revision coupling execution evidence with general repair knowledge.
- [4] AgentFactory: a self-evolving framework through executable subagent accumulation and reuse.
- [5] SoK: Agentic Skills — Beyond Tool Use in LLM Agents, 2026.
- [6] Don’t Break the Cache, arXiv:2601.06007, 2026.
- [7] BAAI. *bge-small-en-v1.5* model card.
- [8] RFC 4122. *Universally Unique IDentifiers (UUIDs)*.